Urban Opportunity, Cost, and Living Conditions Index

```
#   Fix PROJ database conflict before importing geopandas
import os
os.environ['PROJ_DATA'] = '/opt/miniconda3/envs/ps6env/share/proj'
os.environ['PROJ_LIB']  = '/opt/miniconda3/envs/ps6env/share/proj'

import warnings
warnings.filterwarnings('ignore')

import json
import urllib.request
import numpy as np
import pandas as pd
import geopandas as gpd
import altair as alt
import vl_convert as vlc
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

try:
    import seaborn as sns
except ImportError:
    pass  # seaborn not available in this environment
```

Urban Index Construction

Data Loading

```
#   Load census indicator datasets
metro_data = pd.read_csv('data/derived data/metro_data.csv')
pollution  = pd.read_csv('data/derived data/pollution.csv')
rpp        = pd.read_csv('data/derived data/BEA_Metro_RPP_2024_Clean.csv')
emp        = pd.read_csv('data/derived data/Cleaned_MSA_Employment_Growth.csv')

#   Extract CBSA code from GEO_ID (last 5 characters)
metro_data['CBSA'] = metro_data['GEO_ID'].str[-5:]

#   Clean and merge RPP (keep "RPPs: All items" row per CBSA)
rpp['CBSA'] = rpp['GeoFIPS'].str.replace('"', '', regex=False).str.strip()
rpp_all = (rpp[rpp['Description'].str.strip() == 'RPPs: All items']
           [['CBSA', '2024']]
           .rename(columns={'2024': 'regional_price_parity'}))

#   Clean and merge employment (BLS "C1018" → CBSA "10180")
emp['CBSA'] = (emp['area_fips'].str[1:].astype(int) * 10).astype(str).str.zfill(5)
emp_clean = (emp[['CBSA', 'oty_month3_emplvl_pct_chg']]
             .rename(columns={'oty_month3_emplvl_pct_chg': 'employment_growth'}))

#   Merge all datasets on GEO_ID
df = metro_data.merge(pollution[['GEO_ID', 'Median_AQI']], on='GEO_ID', how='left')
df = df.merge(rpp_all,   on='CBSA', how='left')
df = df.merge(emp_clean, on='CBSA', how='left')

print(f"Dataset shape: {df.shape}")
print(f"Columns: {list(df.columns)}")
```

```
Dataset shape: (393, 12)
Columns: ['GEO_ID', 'NAME', 'bachelor_degree', 'graduate_degree', 'median_rent', 'avg_commute
```

Z-Score Standardization

```
#   Helper: compute z-score using (X -  ) /
def z_score(series):
```

```
    return (series - series.mean()) / series.std()

#   Economic Opportunity indicators
df['income_z']            = z_score(df['bachelor_degree'])
df['employment_growth_z'] = z_score(df['employment_growth'])

#   Living Expense indicators
df['rent_z']          = z_score(df['median_rent'])
df['price_parity_z']  = z_score(df['regional_price_parity'])

#   Living Conditions indicators
df['AQI_z']               = z_score(df['Median_AQI'])
df['commute_time_z']      = z_score(df['avg_commute_time'])
df['insurance_z']         = z_score(df['insurance_coverage_rate'])
df['occupants_per_room_z']= z_score(df['avg_occupants_per_room'])
```

Reverse Negative Indicators

Higher z-score should always mean *better* outcome. Reverse indicators where a larger raw value represents a worse situation.

```
#   Reverse: higher raw value = worse outcome
df['rent_z']               = -df['rent_z']           # lower rent = better
df['price_parity_z']       = -df['price_parity_z']   # lower cost = better
df['AQI_z']                = -df['AQI_z']             # lower AQI = cleaner air
df['commute_time_z']       = -df['commute_time_z']    # shorter commute = better
df['occupants_per_room_z'] = -df['occupants_per_room_z']  # less crowded = better
```

Composite Index Construction

```
#   Economic Opportunity Score
df['economic_opportunity_score'] = (
    df['income_z'] +
    df['employment_growth_z']
) / 2

#   Living Expense Score
df['living_expense_score'] = (
    df['rent_z'] +
```

```
    df['price_parity_z']
) / 2

#   Living Conditions Score
df['living_conditions_score'] = (
    df['AQI_z'] +
    df['commute_time_z'] +
    df['insurance_z'] +
    df['occupants_per_room_z']
) / 4

#   Preview
score_cols = ['economic_opportunity_score',
              'living_expense_score',
              'living_conditions_score']
print(df[['NAME'] + score_cols].dropna().head(8).to_string(index=False))
```

```
                                        NAME  economic_opportunity_score  living_expense_sco
                         Akron, OH Metro Area                   -0.053401              0.4025
                        Albany, GA Metro Area                   -0.856164              0.9663
                        Albany, OR Metro Area                    0.084846             -0.7013
      Albany-Schenectady-Troy, NY Metro Area                    1.097320             -0.4460
                   Albuquerque, NM Metro Area                   -0.362025              0.0499
                    Alexandria, LA Metro Area                   -0.107324              1.1250
Allentown-Bethlehem-Easton, PA-NJ Metro Area                    0.827054             -0.6047
                       Altoona, PA Metro Area                    0.095987              0.9225
```

Map Preparation

```
#   Load CBSA shapefile, merge, clip to continental US
cbsa_shp = gpd.read_file('data/map/tl_2023_us_cbsa/tl_2023_us_cbsa.shp')
map_df   = cbsa_shp.merge(df, left_on='CBSAFP', right_on='CBSA', how='left')
map_df   = map_df.cx[-130:-60, 23:52].to_crs('EPSG:4326')
map_df   = map_df.rename(columns={'NAME_y': 'metro_name'})

#   Fetch US state borders as inline GeoJSON
_url = ('https://raw.githubusercontent.com/'
        'PublicaMundi/MappingAPI/master/data/geojson/us-states.json')
with urllib.request.urlopen(_url, timeout=15) as _r:
    states_features = json.loads(_r.read())['features']
```

```
print(f"Map GeoDataFrame: {len(map_df)} CBSA regions")
```

```
Map GeoDataFrame: 917 CBSA regions
```

Visualization Functions

```
def _clean(name):
    if pd.isna(name):
        return ''
    for s in [' Metro Area', ' Metropolitan Statistical Area',
              ' Micropolitan Statistical Area']:
        name = name.replace(s, '')
    return name.strip()


def _table_chart(column, table_title, ascending=False):
    """Altair mark_text top-10 table (pure Altair, no matplotlib)."""
    top10 = (map_df[['metro_name', column]]
             .dropna()
             .sort_values(column, ascending=ascending)
             .head(10)
             .reset_index(drop=True))
    top10['rank'] = [str(i + 1) for i in range(len(top10))]
    top10['city'] = top10['metro_name'].apply(_clean).str[:28]
    top10['row']  = list(range(len(top10)))

    src = alt.Data(values=top10[['rank', 'city', 'row']].to_dict('records'))

    def col(field, w, align, header):
        return (alt.Chart(src)
                .mark_text(align=align, baseline='middle',
                           fontSize=9.5, color='#222222',
                           dx=3 if align == 'left' else
                             (-3 if align == 'right' else 0))
                .encode(y=alt.Y('row:O', axis=None), text=f'{field}:N')
                .properties(width=w, height=310,
                            title=alt.TitleParams(text=header, fontSize=10,
                                                  fontWeight='bold',
                                                  color='#444')))
```

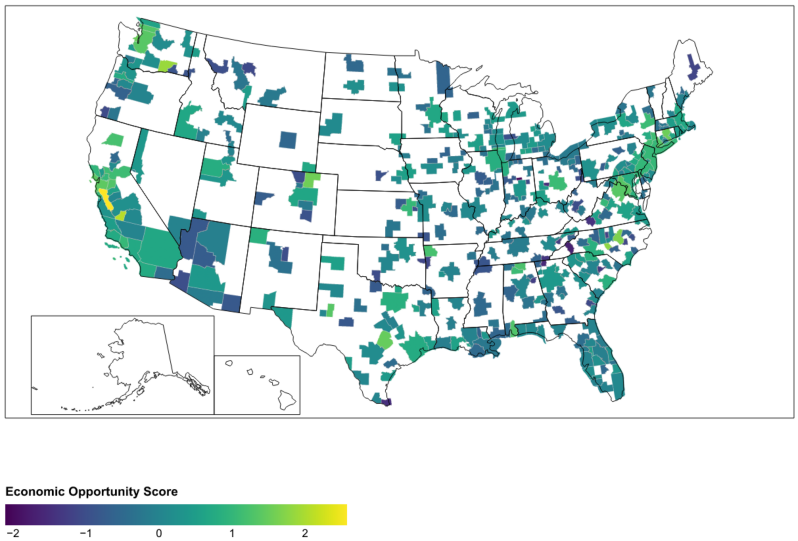
```
    return alt.hconcat(
        col('rank', 22,  'center', '#'),
        col('city', 190, 'left',   'Metro Area'),
        spacing=1
    ).properties(
        title=alt.TitleParams(text=table_title, fontSize=11,
                              fontWeight='bold', anchor='start', dy=-8)
    )


def make_index_chart(column, map_title, table_title, scheme='viridis'):
    """Altair choropleth + state borders + top-10 table (hconcat)."""
    plot_df  = map_df[map_df[column].notna()].copy()
    geo_cols = ['CBSAFP', 'metro_name', column, 'geometry']
    geo_feat = json.loads(plot_df[geo_cols].to_json())['features']

    cbsa_data  = alt.Data(values=geo_feat)
    state_data = alt.Data(values=states_features)

    choropleth = (
        alt.Chart(cbsa_data)
        .mark_geoshape(stroke='white', strokeWidth=0.1)
        .encode(
            color=alt.Color(
                f'properties.{column}:Q',
                scale=alt.Scale(scheme=scheme),
                legend=alt.Legend(title=map_title, orient='bottom',
                                  gradientLength=260,
                                  labelFontSize=8.5, titleFontSize=9.5)
            ),
            tooltip=[
                alt.Tooltip('properties.metro_name:N', title='Metro Area'),
                alt.Tooltip(f'properties.{column}:Q',
                            title='Score', format='.3f'),
            ]
        )
        .project(type='albersUsa')
        .properties(width=600, height=380)
    )

    borders = (
        alt.Chart(state_data)
```

```
        .mark_geoshape(fill=None, stroke='black', strokeWidth=0.5)
        .project(type='albersUsa')
        .properties(width=600, height=380)
    )

    table = _table_chart(column, table_title)

    return (
        alt.hconcat(choropleth + borders, table, spacing=22)
        .configure_view(stroke=None)
    )


def save_and_show(column, map_title, table_title, scheme='viridis'):
    """Generate PNG via vl_convert, then display via matplotlib for PDF."""
    chart = make_index_chart(column, map_title, table_title, scheme)
    png   = vlc.vegalite_to_png(chart.to_json(), scale=2.0)
    path  = f'data/derived data/index_{column}.png'
    with open(path, 'wb') as f:
        f.write(png)

    img  = mpimg.imread(path)
    h, w = img.shape[:2]
    fig, ax = plt.subplots(figsize=(14, 14 * h / w))
    ax.imshow(img)
    ax.axis('off')
    plt.tight_layout(pad=0)
    plt.show()
    plt.close()
```

Economic Opportunity

Higher score indicates higher median earnings and stronger employment growth.

```
save_and_show(
    'economic_opportunity_score',
    'Economic Opportunity Score',
    'Top 10 - Economic Opportunity'
)
```



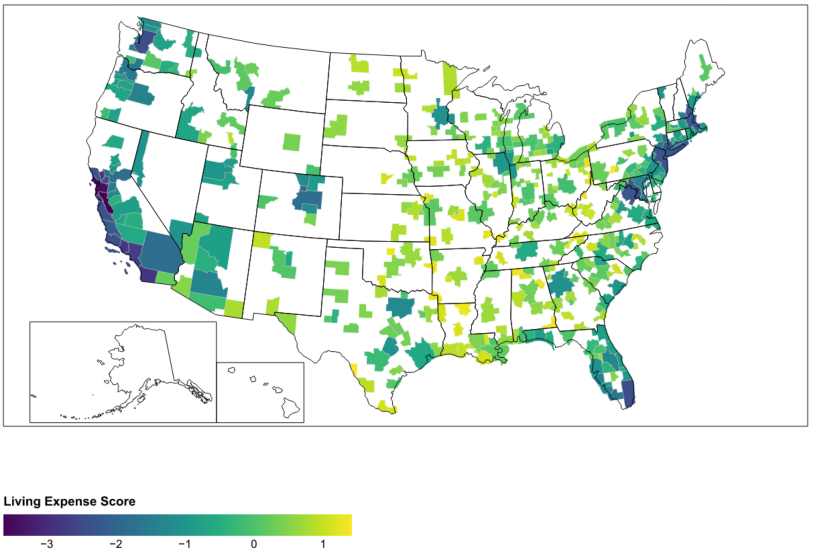
Top 10 — Economic Opportunity

#	Metro Area
1	San Jose-Sunnyvale-Santa Cla
2	Hanford-Corcoran, CA
3	Kennewick-Richland, WA
4	Pinehurst-Southern Pines, NC
5	Raleigh-Cary, NC
6	Greeley, CO
7	Hartford-West Hartford-East
8	Bremerton-Silverdale-Port Or
9	Austin-Round Rock-San Marcos
10	San Francisco-Oakland-Fremon

Living Expense

Higher score indicates lower cost of living (rent and regional price parity).

```
save_and_show(
    'living_expense_score',
    'Living Expense Score',
    'Top 10 - Most Affordable'
)
```



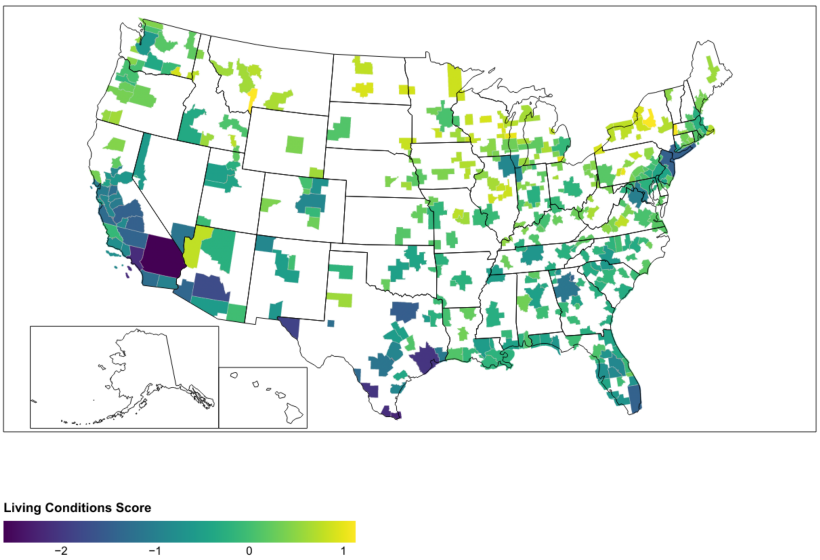
Top 10 — Most Affordable

#	Metro Area
1	Johnstown, PA
2	Eagle Pass, TX
3	Florence-Muscle Shoals, AL
4	Dothan, AL
5	Monroe, LA
6	Anniston-Oxford, AL
7	Gadsden, AL
8	Enid, OK
9	Texarkana, TX-AR
10	Fort Smith, AR-OK

Living Conditions

Higher score indicates cleaner air, shorter commute, better insurance coverage, and less overcrowding.

```
save_and_show(
    'living_conditions_score',
    'Living Conditions Score',
    'Top 10 - Best Living Conditions'
)
```



Top 10 — Best Living Conditions

#	Metro Area
1	Bozeman, MT
2	Utica-Rome, NY
3	Pittsfield, MA
4	Sheboygan, WI
5	Altoona, PA
6	Monroe, MI
7	Bismarck, ND
8	Watertown-Fort Drum, NY
9	Appleton, WI
10	Wausau, WI